MemoKi: Consolidated Progress, Findings and Direction Report

October 2026. Everything done so far, gap by gap: what was built, what was measured, what we gained, and whether the project is heading the right way. Hindi summary at the end.

Status labels

- **Supported**: measured, strong enough for a development-stage claim
- **Partly supported**: measured, with a caveat that limits the claim
- **Not supported**: measured, and the data does not back it
- **Not yet tested**: no experiment has been run

Scope warning. Almost all results are on the **validation split, 100 queries, one seed (11)**. They are development results. The test split (329 queries) is untouched and is reserved for the final run (section 9).

---

1. Summary

MemoKi stores whole conversations in a balanced tree whose nodes carry cheap summaries (names and date spans). A query uses cue checks to skip branches before any vector comparison. **No LLM is used at ingestion or at query time.**

Question	Answer so far
Does it beat brute-force search?	Yes: equal or better recall at 3% to 16% of the comparisons
Do cues really help?	Yes, significantly ($p = 0.012$ against the same tree without cues)
Does the hierarchy beat the best flat method (metadata filter + HNSW)?	**No.** A tie or slightly worse above about 230 comparisons. The tree wins only at very low budgets
Is it faster?	**Not shown.** Name extraction costs about 5 ms per query
Can misfiled conversations be repaired?	Mostly: an offline audit recovers 91% to 100% of lost queries, with small side effects. Query-time fallback fails
Does a recency rule fix outdated facts?	Pilot only: harmless, moves pairs in the right direction, **not significant** (25 pairs)
Against LLM-based memory (Mem0)?	**0 LLM calls** at ingestion and search against 1 call per session for Mem0; about 1 minute against about 5 hours of ingestion (extrapolated). Accuracy not compared

Direction in one line: the process is sound and the claims are honest. The defensible contribution is **LLM-free, cue-guided hierarchical memory with a measured cost advantage over LLM-based memory**, not "faster than every flat method".

---

2. Setup

Item	Value
Data	LongMemEval-S: 19,195 sessions, 470 answerable questions (30 abstention questions excluded). LoCoMo-10: 272 sessions, 5,882 turns, about 182,000 tokens, 1,986 questions
Split	141 validation / 329 test queries, seeded and saved
Stores	1,000, 5,000, 10,000 sessions (500 in early runs), nested
Embeddings	all-MiniLM-L6-v2, 256 tokens, computed once for all methods
Metric	Recall@5; cost counted as embedding comparisons (cue checks counted separately)
Hardware	CPU laptop, 8 cores, 7.8 GB RAM, no usable GPU
Code	81 automated tests exist (75 confirmed passing on your machine; the later additions passed in my environment)

---

3. What was built

Part	What it is
Foundation	One config, shared cost counters, common retriever interface, fixed split, resumable caches
Cues	Name extraction (spaCy) and a rule-based date parser. No LLM
Tree	Balanced hierarchy (32 children per node, height 3 at 10,000), structure chosen by a rule written before the results
Cue-guided search	Name and date summaries on nodes, cue gate before vector comparison
Opponents	Brute force, FAISS flat, HNSW, flat + cue filter, filter + HNSW hybrid
Gap 2 tools	Misfiling simulator, three query-time recovery tiers, offline audit and re-filing
Gap 3 tools	Update-pair builder, recency rule (two variants), gap diagnostic, bias check
Gap 4 tools	Ollama timing, Mem0 spike with an LLM-call meter, our-method counterpart on the same slice

4. Gap 1: cost against scale

Recall@5 / mean comparisons per query (validation, beam 10):

Method	1,000 sessions	5,000 sessions	10,000 sessions
Brute force	0.630 / 1,000	0.460 / 5,000	0.360 / 10,000
HNSW (ef 16)	0.640 / 255	0.450 / 316	0.350 / 320
Filter + HNSW (ef 16)	0.710 / 162	0.520 / 243	0.440 / 297
Tree, no cues	0.600 / 254	0.390 / 463	0.330 / 457
Cue tree	**0.680 / 162**	**0.470 / 303**	**0.420 / 308**

Findings

- Brute-force cost grows linearly and its accuracy falls with size (0.63, 0.46, 0.36). This is the problem the project targets.
- **Cues help significantly:** cue tree against the same tree without cues, 10 wins and 1 loss ($p = 0.012$).

- **Queries with a cue (41) against without (59), 10,000 sessions:** cue tree 0.634 against brute force 0.488 on cue queries. Queries without cues are unchanged by design (0.271 for all methods).
- **Low budget:** the cue tree reaches 0.350 with 108 comparisons, where plain HNSW needs about 320 for the same recall. This is the tree's niche.
- **Above about 230 comparisons,** filter + HNSW matches or beats the cue tree (0.440 at 297 against 0.420 at 308; 2 against 4 queries, p = 0.69).
- **HNSW also scales sub-linearly** (255, 316, 320 comparisons), so "flat search grows linearly" holds for brute force only.
- **Ablation (beam 10):** no cues / names / names + time: 0.330 / 0.410 / 0.420 at 10,000 sessions. Time adds about 1 point.
- **Cue quality:** names appear in 36% to 37% of queries, with recall 97% (36/37) and selectivity 0.9% on validation (95% on the earlier test-query check). Dates apply to 8% of queries, with recall 11/11.
- **Time:** search takes about 0.27 ms against 0.98 ms for brute force at 10,000 sessions, but name extraction adds about 5 ms per query, so end to end the cue tree is slower than HNSW (0.06 to 0.15 ms).

What we gained: a measured, honest cost curve; a significant cue effect; a clearly identified niche. **Verdict: partly supported.**

---

5. Gap 2: misfiled conversations

Damage (5,000 sessions, beam 10, 5 repeats, clean recall 0.470):

Misfiled	Recall@5	Lost queries	Cue / no-cue
5%	0.468	4	0 / 4
10%	0.450	12	4 / 8
20%	0.432	25	6 / 19

Cue summaries protect partly: cue queries are 41% of queries but only 24% to 33% of the losses.

Query-time recovery fails the synopsis criterion (over 80% recovery with under 25% extra cost). At 10% misfiling the options were: entity lookup 4/12 recovered at +5.7% cost; wider beam x8 10/12 at +397%; full flat 12/12 at +1,642%, breaking 26 correct queries. The "weak similarity" trigger does not separate lost queries from normal ones.

Offline audit (margin frozen at 0.10 by the amended rule): recovers 91% to 100% of lost queries at all three sizes, with query cost within about 2%.

Size	Misfiled	Recovered	Harmed	Recall, misfiled then audited
1,000	10%	5 of 5	3	0.676, then 0.690
5,000	10%	11 of 12	4	0.450, then 0.484
10,000	10%	10 of 11	17	0.398, then 0.400

Side effects: it re-files 10% to 15% of healthy leaves, harms some other queries, and its net recall gain ranges from -0.8 to +4.2 points (about zero at 10,000 sessions). Cost is quadratic in store size (4.75 million comparisons at 10,000 sessions, offline). It does **not** reliably improve a healthy tree (+1, +2, -1 points at the three sizes, not significant).

Still to do: the hand-labelled cue-accuracy sheet (not filled in), and the test split. **What we gained:** a measured cost of misfiling, and a repair that works without query-time cost. **Verdict: largely answered on development data.**

---

6. Gap 3: outdated information

Pilot: 25 update pairs (LongMemEval knowledge-update queries; both the older and newer session are evidence).

(5,000 sessions)	Brute force	Cue tree
Older above newer, top 5	16%	20%
Neither session in the top 5	68%	56%
Both in the top 50	10 of 25	8 of 25
Older above newer, among those	70%	50%
Best rule, post-hoc metric	70% to 60% (1 pair of 10)	**50% to 25%** (2 pairs of 8, p = 0.5)

Findings

- The rule (bonus for the newer conversation among near-tied candidates) is free, adds no comparisons, and costs no recall at small settings. But no effect is significant: 25 pairs, and for most pairs neither session is even retrieved.
- **Diagnostic:** the older session scores higher in 12 of 25 pairs and the newer in 13 (median gap -0.007). Of the older-higher pairs, 6 of 12 are near-ties within 0.05, so a small bonus has real headroom.
- **My first version of the rule had a flaw:** it gave a bonus only to candidates near the top score. The faithful "close to each other" variant (`local`) did better.

- **Bias check:** evidence sessions sit at mean recency percentile 0.48, so no timestamp artifact explains the small recall gains (+0.011 to +0.043, that is 1 to 4 queries).
- **Retrieving knowledge-update evidence is itself hard** with one vector per whole session: the newer session was in the top 5 for only 20% to 36% of pairs.
- Disclosure: the pairwise analysis and the `local` variant were added **after** seeing the first run. Both are reported as post-hoc.

Frozen for the test split: $\delta = 0.01$, $\lambda = 0.01$, both `top` and `local` variants, no selection on test. **What we gained:** a working LLM-free rule, a diagnostic that says what it can and cannot fix, and a finding about update retrieval. **Verdict: pilot, inconclusive but harmless.**

---

7. Gap 4: fair comparison

Measured so far

Item	Result
Local LLM speed (llama3.2:3b, CPU only)	about 6.6 to 8.4 tokens/s generated, 47 to 57 tokens/s prompt processing; model 2.6 GB; memory 7.4 of 7.8 GB in use
Mem0 2.2.1, 3 sessions (58 turns)	3 LLM calls (1 per add), 6,150 prompt + 602 output tokens, 3.2 min; 0 LLM calls per search, 35 ms
Mem0 extrapolated (linear)	about 304 calls and about 5 hours for all of LoCoMo on this laptop
Our method, same slice	0 LLM calls, 0.60 s ingestion (about 61 s extrapolated), search about 5.7 ms
Our evidence hit rate, top 1 / 2 / 3 sessions	72% / 96% / 100% (3-session slice, so top 3 is trivial)
Tokens retrieved, ours	about 612 / 1,377 / 2,244 for 1 / 2 / 3 sessions
Tokens retrieved, Mem0	about 285 for 14.5 memories (being re-measured at 5 memories)

Open issues with the Mem0 numbers

1. The prompt size was **exactly 2,050 tokens in all three adds** despite different session lengths, which suggests truncation at a 2,048-token context. A diagnostic is built; the result is pending. If confirmed, Mem0 saw only part of each session.
2. `limit=5` appears to have been ignored (14.5 memories returned). The script now cuts results to 5 on our side.
3. Mem0 ran in a **reduced mode** (no keyword search, no entity component) with a **3B local model**.

4. Accuracy is **not measured** for either system. Our retrieved-token cost is higher because we return whole sessions; Mem0 returns short facts.

Foundation (done): one config, same data, queries, embedding model, seeds and split; shared cost counters; rules written before results; paired exact tests; partial ablation (no cues / names / names + time / fallback). **Not done:** HiMem; type and emotion cues; full ablation table; repeats over 3 to 5 seeds; the test split. **What we gained:** a measured cost advantage over LLM-based memory that matters to a real user, and a practical limit on what the laptop can run. **Verdict: foundation solid; the comparison is partly measured.**

---

8. Claims and validation

#	Claim	Evidence	Status
C1	Matches brute-force accuracy at a small share of the cost	0.68 / 0.47 / 0.42 vs 0.63 / 0.46 / 0.36 at 16% / 6% / 3% of comparisons	**Supported** (dev)
C2	Cues significantly improve the tree	10 wins, 1 loss, p = 0.012	**Supported**
C3	Better than brute force on queries with cues	0.634 vs 0.488; p = 0.109	Partly supported
C4	Beats flat search including HNSW	Hybrid filter + HNSW equal or better; p = 0.69	**Not supported**
C5	Best at very low budgets	0.350 at 108 comparisons vs HNSW 320	Partly supported
C6	Cost grows slowly with size	162, 303, 308 comparisons; HNSW is also flat	Supported, not unique
C7	Faster than flat search	About 5 ms name extraction per query	**Not supported**
C8	Zero LLM calls	Counter is 0 at ingestion and search	**Supported**
C9	Misfiling costs recall	-0.2, -2.0, -3.8 points at 5%, 10%, 20%	Supported (small counts)
C10	Offline audit repairs misfiling	91% to 100% recovered; net effect -0.8 to +4.2 points	Partly supported
C11	Query-time fallback repairs misfiling	No setting passes the criterion	**Not supported**
C12	Audit improves a healthy tree	+1, +2, -1 points; not significant	**Not supported**
C13	Name cue is reliable	Recall 97% (36/37), selectivity 0.9%	**Supported** (dev); precision pending
C14	Date cue is precise but rare	8% of queries; recall 11/11	Partly supported
C15	Recency rule reduces stale retrieval	50% to 25% (2 pairs of 8, p = 0.5); harmless	Pilot, not significant

#	Claim	Evidence	Status
C16	Update evidence is hard to retrieve	Newer session in top 5 for 20% to 36% of pairs	Supported (descriptive)
C17	No LLM cost at ingestion, unlike Mem0	0 vs 1 call per session; about 1 min vs about 5 h (extrapolated)	**Supported** on a slice; extrapolated
C18	Retrieves fewer tokens than Mem0	Ours 612 per session; Mem0 about 285 for 14.5 memories	**Not supported** (we expect to lose)
C19	Comparison with HiMem	Not run	Not yet tested

9. Validation hygiene

What protects the results: fixed saved split; rules written before results (tree structure, fallback threshold, audit margin, recency selection); paired exact tests; counts rather than milliseconds as the cost measure; strong opponents; bugs found and fixed (cache loss, parser crash, stale cue cache, negative search time, model-load inflation, CSV columns, a wrong rule design in Gap 3).

What weakens them (state in the final report)

1. 100 queries and one seed: one query is 1 point, and many differences are 1 to 4 queries.
2. Small denominators (Gap 2 lost counts 4, 12, 25; Gap 3 pairs 8 to 25), and repeats reuse the same queries.
3. Early checks used test queries (first baseline, model comparison, first entity check). The entity check was since replicated on validation.
4. Two post-hoc changes: the audit margin rule was tightened, and Gap 3 gained a second analysis and rule variant.
5. Linear extrapolation from a 3-session slice; Mem0 in reduced mode with a 3B model; possible context truncation unresolved.
6. My misfiling simulation is easier than natural misfiling.

10. Are we going in the right direction?

Yes, with a changed headline.

What is right	Why
The method	Every claim is measured against strong opponents, with rules written before results. That is what makes the findings defensible
The honesty	The synopsis claim "the tree beats flat search" failed against filter + HNSW, and the report says so
The strongest contribution	**LLM-free memory with measured cost advantages**: zero LLM calls and about a minute of ingestion against hours for LLM-based memory, plus significant cue gains, a quantified misfiling problem with a working repair, and a recency pilot

What to adjust	Action
The claim	Reframe around LLM-free, cue-guided retrieval and its cost, not "faster than flat search". Show the mentor the Pareto plots and tables soon
The comparison	Report retrieved-token cost honestly (we lose), and state that accuracy was not compared with Mem0 or HiMem
The evidence	Everything needs the test split, several seeds and confidence intervals before it becomes a final claim

Risks: HiMem may be impractical on this laptop; type and emotion cues may be weak or dropped; the Mem0 truncation question may change its numbers; the browser extension is exposed to chat-site changes.

11. What remains

Order	Work	Purpose
1	Re-run the 1-session Mem0 diagnostic; fix context if truncated; re-measure at 5 memories	Make the Mem0 numbers sound
2	Run our method on all 19 sessions of conv-26 (`eval.locomo_ours --sessions 19`)	A realistic hit rate
3	HiMem feasibility check	C19
4	Hand-label 100 conversations (`eval.label_sample`, `eval.score_labels`)	Finish Gap 2 cue accuracy
5	Decide on type and emotion cues (test quickly, or drop with reasons)	Gap 4 ablation
6	**Final test-split run**: 3 to 5 seeds, five sizes, more queries, confidence intervals, all frozen settings (audit margins, recency candidates)	Final evidence
7	Optional: 50k to 100k stress test; cheaper name extraction	C6, C7
8	Server, extension, dashboard (minimal)	Delivery
9	Write-up	Submission

My estimate is that **about 60%** of the project is done by effort. It is an estimate, not a measurement.

12. Reproducing the results

Result	Command	Saved in `results\`
Tests	`pytest`	
All methods side by side	`python -m eval.compare_check --sizes 1000 5000 10000 --efs 5 8 16 32 64 128 --beams 1 2 3 5 10 20 --audit-margin 0.0 --csv results\pareto_audit.csv`	`pareto_audit_table.txt`, `pareto_*.png`
Misfiling and recovery	`python -m eval.misfile_check --sizes 5000`	`misfile_5000.txt`
Offline audit	`python -m eval.refile_check --sizes 1000 5000 10000`	`refile_5000.txt`, `refile_1000_10000.txt`
Recency pilot	`python -m eval.recency_check --csv results\recency_5000.csv`	`recency_5000.txt`
Our method on LoCoMo	`python -m eval.locomo_ours --sessions 3`	`locomo_ours_3.txt`
Mem0 spike (in `.venv-mem`)	`python -m spike.mem0_spike --sessions 3 --granularity session`	`mem0_spike.json`

---

13. हिंदी में सार

क्या बना: पूरी नींव, नाम और तारीख के संकेत, संतुलित tree, संकेतों से चलने वाली खोज, मज़बूत opponents (brute force, HNSW, filter + HNSW), Gap 2 के औज़ार, Gap 3 का recency नियम, और Mem0 से तुलना की शुरुआत। कुल 81 tests हैं।

हर gap का नतीजा (development data पर):

- **Gap 1:** brute force से बराबर या बेहतर नतीजा सिर्फ़ 3% से 16% तुलनाओं में। संकेत सचमुच काम करते हैं ($p = 0.012$)। लेकिन "filter + HNSW" लगभग बराबर या थोड़ा बेहतर है, और समय का कोई दावा नहीं बनता (नाम निकालने में लगभग 5 ms लगते हैं)।
- **Gap 2:** galat shakha में बातचीत जाने से recall 2 से 4 अंक गिरता है। Query के समय fallback असफल रहा। Offline audit ने 91% से 100% खोए सवाल ठीक किए, कुछ साइड इफ़ेक्ट के साथ।
- **Gap 3:** recency नियम मुफ़्त और बेनुकसान है और सही दिशा में असर डालता है (50% से 25%), पर सिर्फ़ 25 जोड़ों पर यह पक्का नहीं हुआ। एक और बात: नई जानकारी वाली बातचीत ढूँढना ही मुश्किल है।

- **Gap 4:** हमारे तरीके में LLM की कोई call नहीं (0), जबकि Mem0 में हर session पर 1। हमारा ingestion लगभग 1 मिनट, Mem0 का लगभग 5 घंटे (अनुमान)। लेकिन retrieved tokens में Mem0 जीतेगा, और सटीकता (accuracy) की तुलना नहीं हुई है। HiMem अभी नहीं चला।

सही दिशा? हाँ। तरीका बहुत मज़बूत है और दावे ईमानदार हैं। बस मुख्य दावा बदलना होगा: "tree सबसे तेज़ है" के बजाय "बिना LLM के, संकेतों वाली याददाश्त, जिसका खर्च LLM वाली याददाश्त से बहुत कम है"। Mentor को Pareto plots और tables जल्दी दिखाइए।

सावधानियाँ: अभी 100 सवाल और एक seed है, इसलिए छोटे अंतर भरोसे लायक नहीं। असली नतीजे के लिए test split पर कई seeds के साथ दोबारा चलाना होगा। Mem0 के आँकड़ों में context कटने का शक बाकी है।

अनुमानित प्रगति: लगभग 60% (यह अनुमान है, नापी हुई संख्या नहीं)।